

BANZA: An Open Protocol for Financial Interoperability

Fidel R. Monteiro¹ and Jesus R. Monteiro¹

¹BANZAMI – Tecnologia e Serviços, Lda., Luanda, Angola
Official website: <https://banza.network>

Correspondence: Fidel R. Monteiro (contact@banza.network)

Abstract. Independent financial operators interoperate through bilateral integrations and shared infrastructures, but the specifications, the tests and the results are not always public, which makes the technical validation hard to reproduce. This article presents BANZA, an open financial interoperability protocol based on public, versioned rules, deterministic validation and verifiable evidence. The model evaluates implementations against public versions and profiles, separates the protocol, certification and operational schemes, and binds each result to the inputs that produced it through canonical artifacts, cryptographic digests, reason codes and receipts. In this way, technical conformance can be inspected, reproduced and verified by third parties. The protocol and the reference implementation remain in pre-production; performance, scalability, adoption and behaviour in real operational environments have therefore not yet been demonstrated experimentally.

1 Introduction

Financial operators rarely function in isolation. To exchange value or messages, their implementations must agree on formats, identity mechanisms, keys, error semantics and test procedures. In bilateral integrations, these elements are defined between each pair of operators; when specifications, tests and results remain private, work may be repeated and the technical validation becomes hard for third parties to reproduce.

Let n be the number of operators. Equations (1a) and (1b) represent, respectively, a mesh of bilateral integrations and the adoption of a common protocol.

$$R_{\text{bilateral}}(n) = \frac{n(n-1)}{2} \tag{1a}$$

$$I_{\text{comum}}(n) = n \tag{1b}$$

Equation (1a) counts the technical relationships between pairs of operators, whereas Equation (1b) counts the implementations required when each operator adopts the common protocol. The entry of a new operator may therefore require up to n new bilateral integrations, but only one new implementation of the protocol. Figure 1 illustrates the two regimes.

**The Portuguese-language edition is the canonical version of the BANZA Whitepaper. The English-language edition is an official translation. In the event of an unintended divergence, the canonical Portuguese edition prevails.*

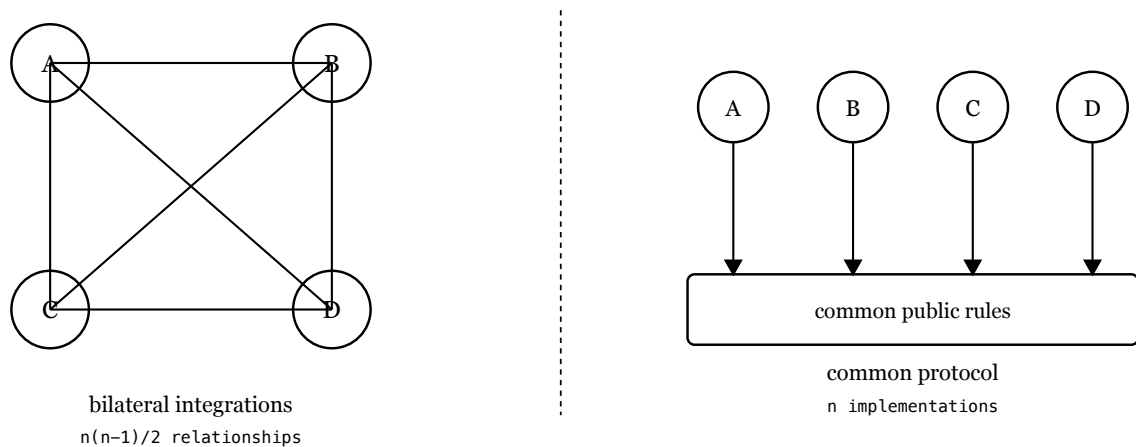


Figure 1. Comparison between a full bilateral mesh, with $n(n-1)/2$ technical relationships, and a common protocol, with n implementations, one per operator.

Interoperability can also be provided by shared, centralised infrastructures. These models enable operation between participants, but their specifications, tests and results are not always available in a form that permits independent reproduction. BANZA does not seek to replace this operational interoperability: it adds an open layer of public, versioned specifications, profiles and contracts, deterministic conformance and verifiable, reproducible evidence.

Standards such as ISO 20022 [1], ISO 8583 [2], the EMV specifications for QR payments [3] and ISO/IEC 9646 [4] address relevant components of messaging, payments and conformance testing. BANZA complements this work by defining an open protocol in which each implementation publishes artifacts at its canonical origin, is evaluated against a public version and profile, and produces results bound to the observed inputs. Evaluation bears on implementations, not on entities, keeping the protocol, certification and operational schemes separate.

This document is foundational and non-normative. §2 and §3 present the BANZA model and architecture; §4, the conformance profiles; §5, the discovery, identity and trust mechanisms; and §6 and §7, validation and evidence. Security and governance are addressed in §8 and §9; the limitations and the current state, in §10 and §11; and §12 presents the conclusions.

2 Model

BANZA distinguishes an operator from an implementation. An operator is the responsible organisational entity; an implementation is the technical system being evaluated, and one operator may publish several implementations — demonstration, test, pre-production or production. Any validation or certification result always applies to a specified implementation, protocol version, profile, environment, scope, evidence set and validity period, and never to an entity in the abstract. Each implementation has a stable identifier, and the exact set of artifacts evaluated — the build — is pinned by a cryptographic content digest. A different build is a distinct subject of evaluation, even if it belongs to the same implementation, and requires its own evaluation.

We model an implementation as a tuple of six elements. In Equation (2), o identifies the operator, i the implementation, v the protocol version, p the conformance profile, e the environment and u the canonical origin.

$$\mathcal{I} = (o, i, v, p, e, u) \quad (2)$$

The artifacts observed at the origin at a given instant t form a set. Equation (3) represents that set, where n is the number of artifacts.

$$A_t(\mathcal{I}) = \{a_1, a_2, \dots, a_n\} \quad (3)$$

Validation takes those artifacts and the applicable specification S for version v and profile p . Equation (4) describes this process. The evaluation is performed by engine version m and produces the result R , the evidence E and the receipts P . In this model, R comprises the verdict and the reason code that grounds it.

Scope is therefore explicit rather than implied: a result states the profile it was evaluated against, the protocol version it assumed, the environment of the execution and the evidence it consumed. It is valid only for that combination and never generalises silently from the evaluated artifact to the entity that published it. Reproducibility is a first-class property: given equivalent canonical inputs, the same specification, the same profile and the same engine version, independent executions produce semantically equivalent verdicts and reason codes [8]. Non-deterministic metadata, such as execution timestamps, is excluded from this equivalence.

$$V_m(A_t(\mathcal{I}), S_{v,p}) \rightarrow (R, E, P) \quad (4)$$

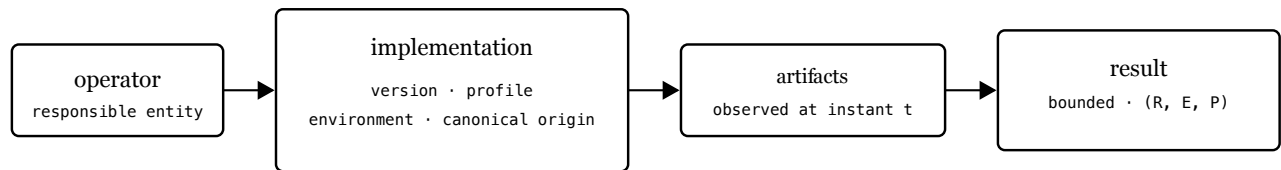


Figure 2. An operator publishes an implementation with a version, a profile, an environment and a canonical origin; the artifacts observed at that origin are evaluated and yield a result valid only for that combination.

Cryptographic content digests, computed with SHA-256 [5], make it possible to identify and compare the observed inputs exactly. Together with the availability of those inputs, they turn it was validated into anyone can re-validate it. Figure 2 relates the operator, the implementation and its attributes to the observed artifacts and the bounded result.

3 Architecture

BANZA is organised into three institutional layers, separated by responsibility, infrastructure and keys. Layer 1 corresponds to the open protocol and brings together the common rules, contracts and mechanisms of discovery, identity, trust, revocation and federation. Layer 2 evaluates implementations against public profiles and versions through deterministic engines in Rust. Layer 3 comprises independent operational schemes that may adopt BANZA and remain subject to the applicable legal and regulatory framework.

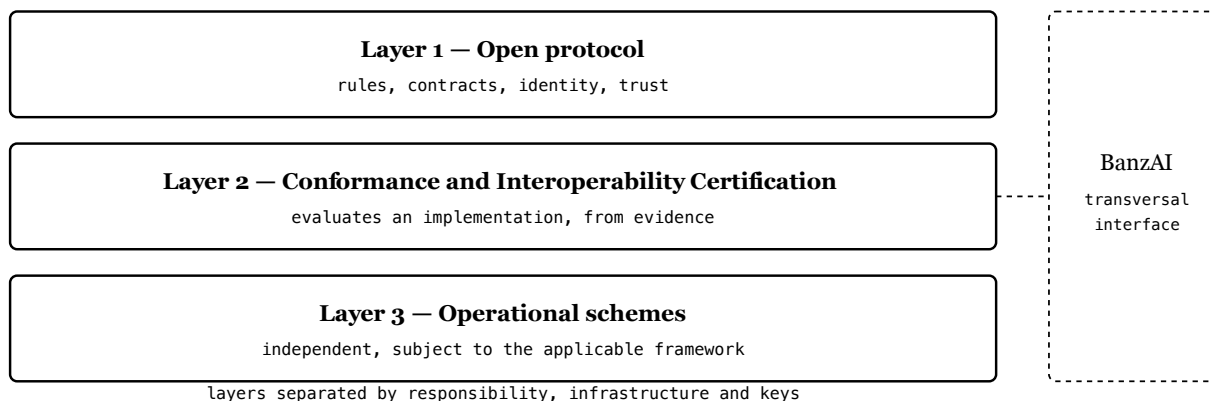


Figure 3. Layer 1, open protocol; Layer 2, per-implementation certification; Layer 3, independent operational schemes. BanzAI is a transversal interface across all three, not a fourth layer or an authority.

The separation of the layers preserves the protocol’s neutrality: every implementation is evaluated against the same profiles, engines and evidence model. Technical certification, admission to a scheme and regulatory authorisation are independent determinations. BanzAI serves as the human interface for consulting and explaining results, without taking part in the decision; machine consumers reach the public interfaces directly. Figure 3 summarises this architecture. The profiles L0–L4, presented in §4, are distinct from the three layers.

4 Profiles

BANZA defines five cumulative conformance levels, L0 to L4. Technical adoption proceeds by selecting a profile, publishing the artifacts at the canonical origin and executing the validation journey. A passing result at a level constitutes technical evidence of interoperability, not a certification, authorisation or approval.

Level L0 — Protocol Sandbox — validates the secure configuration of the protocol in a test environment, with a valid manifest and a correct monetary representation. L1 — Core Payment Capability — adds essential payment and traceability capabilities. L2 — Payment Initiation Capability — adds payment initiation by request or by dynamic QR code. L3 — Inter-Operator Interoperability — adds interoperability between operators, including settlement and reconciliation, constitutes the

eligibility threshold for federation and requires multi-operator evidence. L4 — External Interoperability — adds integration with external networks; it is profile-defined and requires specific evidence of that integration.

Levels L0 to L2 can be assessed within a single operator, whereas L3 and L4 require, respectively, federation evidence and external-integration evidence. A certification profile is distinct from these levels: it pins one conformance level and the set of required capabilities, schemas, contracts and endpoints. Figure 4 summarises the cumulative progression and what each level adds.

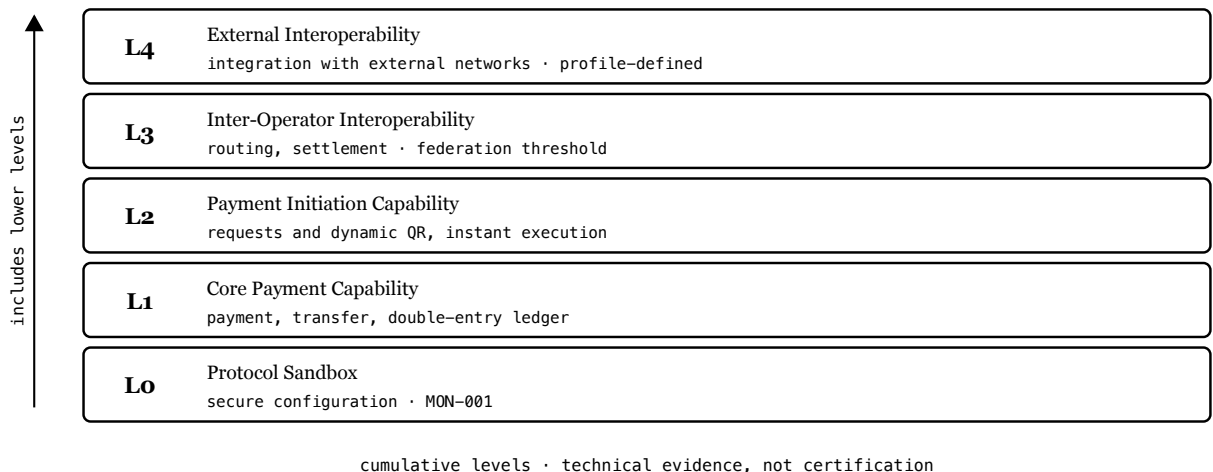


Figure 4. Each level includes all lower ones and adds capabilities; L3 is the federation eligibility threshold and L4 is profile-defined, with external integration evidence.

5 Discovery

Each implementation publishes its artifacts at a canonical origin controlled by the operator [7]. Discovery begins at a fixed path under that origin and returns a Manifest with the identification of the implementation, the endpoints, the protocol version, the declared profile, the signed metadata and the public keys. The Technical Registry associates the operator, the implementation and the canonical origin, and every announced endpoint must belong to that origin.

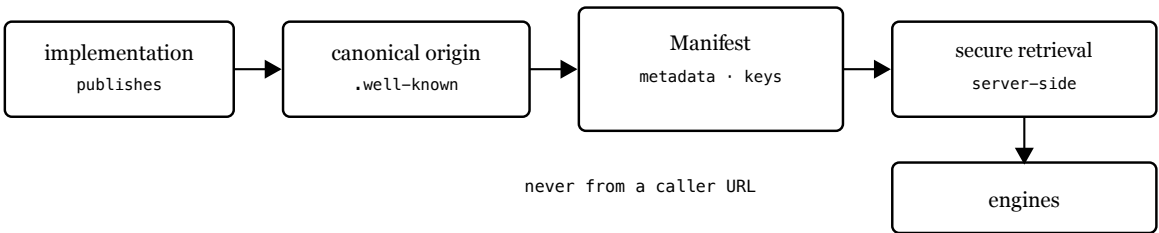


Figure 5. An implementation publishes signed artifacts at the canonical origin the operator controls; a server-side secure retrieval component obtains the Manifest and keys and passes them to the engines, never from a caller-supplied URL.

Identity and integrity rest on published keys and signed metadata. A trust root, kept offline, signs the Key Manifest with Ed25519 [6]; that signature is the basis for trusting the issuing keys. Keys are separated by domain, and revocation makes it possible to detect revoked, superseded or expired material.

The engines retrieve the artifacts server-side through a secure retrieval component, which resolves the target from the registry and never follows a caller-supplied URL. Figure 5 summarises this path.

6 Validation

Validation follows a fixed journey of eight technical steps and one final aggregation step. Each step is executed by deterministic engines in Rust and receives one of four statuses: verified, pending, failed or blocked. The evaluation fails closed: absent, expired or inconsistent evidence never produces a pass. Certification readiness is reached only when every technical step is verified, but it does not, by itself, constitute a certification.

Decision remains separate from explanation. The deterministic engines produce the results, while BanzAI only presents and explains them, without altering or deciding them. Technical certification is a distinct process, which formally binds an implementation to a public, versioned profile. Figure 6 summarises the journey, the statuses and the separation.

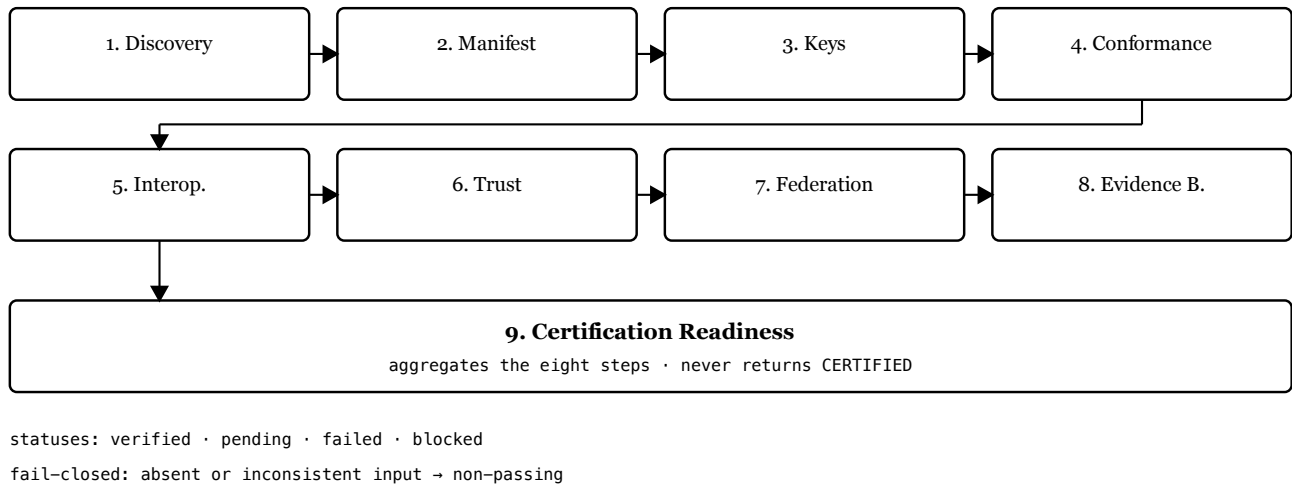


Figure 6. Eight technical steps and one aggregation step; each step receives a status with reason codes and fails closed; Certification Readiness is reached only when every technical step is verified and never returns a certification verdict.

Figure 7 shows an example between two operators: B publishes its artifacts, A initiates the evaluation and the engines produce results, receipts and verifiable evidence. BANZA provides the technical basis for the decision, but it is for Operator A to decide, under its own policy, whether the result meets its needs.

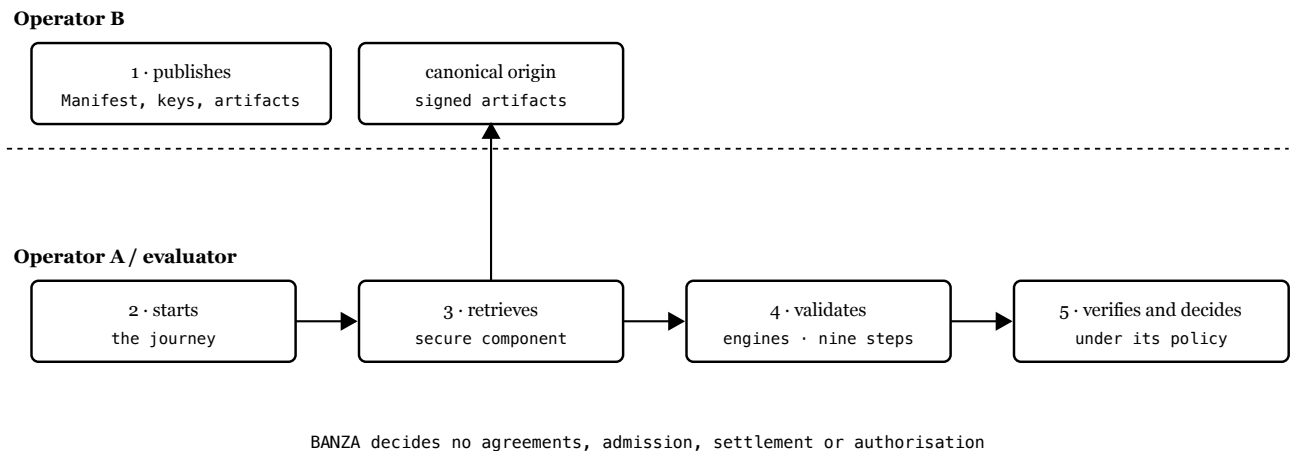


Figure 7. Operator B publishes the artifacts at its canonical origin; Operator A, or an evaluator, starts the journey; the secure component retrieves the artifacts, the engines validate and produce results, receipts and evidence; Operator A verifies or reproduces the result and decides, under its own policy.

7 Evidence

Each validation produces results, receipts and verifiable evidence. Receipts record the executed steps and bind each result to the protocol version, the profile, the artifacts consumed, their cryptographic digests and the reason codes, making it possible to confirm or reproduce the technical basis of the evaluation (Figure 8).

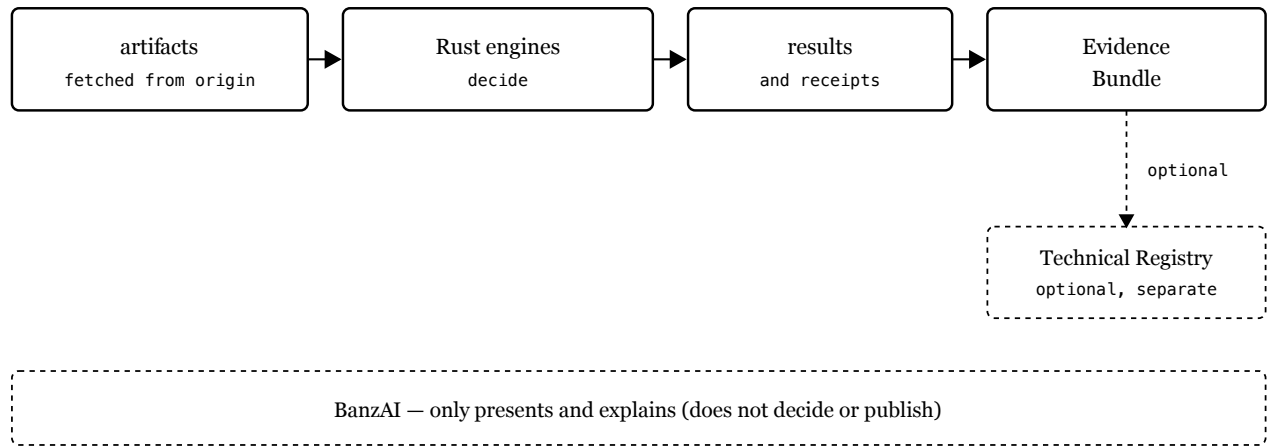


Figure 8. Artifacts enter the engines, which decide and emit results; the results become receipts and an Evidence Bundle; publication in the Technical Registry is optional and separate; BanzaI only presents and explains.

A receipt represents the state observed at the instant of evaluation; later changes to the artifacts require a new evaluation. The Evidence Bundle gathers the artifacts and references that support the result, but it does not constitute certification and grants no operational status.

Publication is separate from evaluation. Results may be published in the Technical Registry, a public, read-only surface that indexes verifiable metadata and evidence. The absence of an entry is not a prohibition, and any third party can re-run the evaluation from the published artifacts.

8 Security

The threat model considers tampering with artifacts, invalid origins or keys, replay of challenges, expired or incomplete evidence and version divergences. For remote retrieval, it further includes SSRF and DNS rebinding. BANZA responds with signatures, revocation, expiry, single-use challenges and fail-closed evaluation, in which absent or inconsistent inputs are never passed (Figure 9).

threat		mechanism		expected outcome
artifact tampering	→	signatures	→	detection
false origin	→	resolved canonical origin	→	rejection
revoked/expired key	→	revocation and expiry	→	non-passing
challenge replay	→	single-use challenge	→	refusal
SSRF · DNS rebinding	→	hardened retrieval	→	blocking

when evidence is missing or inconsistent, evaluation fails closed				

Figure 9. Each considered threat corresponds to a protection mechanism and an expected outcome; absent or inconsistent inputs lead to non-passing, by failing closed.

The secure retrieval component is the only component that reaches the operator’s endpoints. The target is always resolved from the registered canonical origin, never from a caller-supplied URL. The component accepts only HTTPS, blocks private, local and cloud-metadata addresses, pins the connection to the validated addresses and follows no redirects. It also validates TLS and caps the received responses.

9 Governance

BANZA evolves through public decision instruments: ADRs record adopted decisions and RFCs propose changes still under discussion. The protocol follows semantic versioning; compatible changes enter minor or patch versions, whereas incompatible changes require a new major version. Each implementation declares the version and the profile it adopts, and a certification profile is immutable once published.

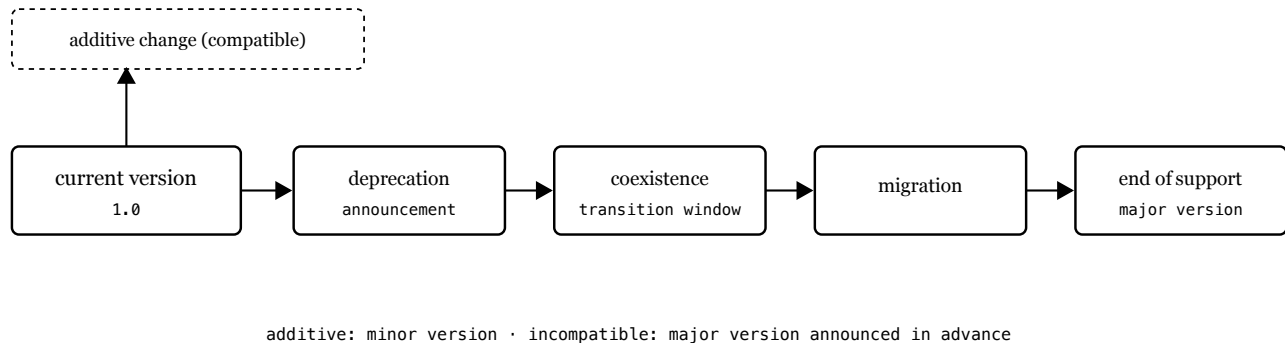


Figure 10. A current version may receive additive, non-breaking changes; an incompatible change opens a major version, announced as a deprecation, with coexistence during a transition window, migration and, finally, end of support.

Incompatible changes are announced in advance and accompanied by a migration path, allowing temporary coexistence between versions. Deprecation is explicit and phased: a feature is first marked as deprecated and can only be removed in a major version (Figure 10). The current protocol version is 1.0.

10 Limitations

BANZA’s guarantees have clear boundaries. First, they are technical, not regulatory: a passing result demonstrates conformance to a public profile, but does not imply authorisation to operate.

Second, reproducibility applies only to the deterministic engines over the declared inputs. It does not extend to properties the protocol does not observe, such as an operator’s internal controls or off-protocol behaviour. A compromised origin or key can only be addressed once there are observable signs or a declared revocation.

Finally, the evidence presented in this document is architectural. It demonstrates the mechanism on a reference implementation, but does not measure performance, scale or adoption, which remain questions for empirical study (Figure 11).

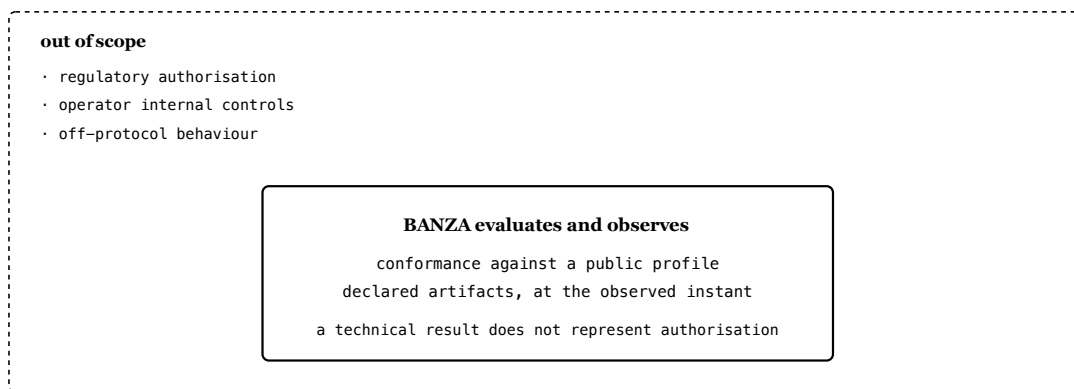


Figure 11. BANZA evaluates the conformance of an implementation against a public profile and observes the declared artifacts; regulatory authorisation, internal controls and off-protocol behaviour remain out of scope, and a technical result represents none of these.

11 State

BANZA remains in pre-production. At the time of publication, the Technical Registry contains zero production operators and zero active technical certifications; real-money payments remain disabled while the operational layer is in regulatory preparation. This state is enforced by a fail-closed control in Rust. No performance measurements are reported, and the empty registry corresponds to the expected current state (Figure 12).

The reference implementation, Operator Zero, runs in an isolated test environment, in read-only mode and with a demonstration currency. It moves no real funds, remains not certified and not intended for production, and is evaluated by the same engines and the same journey applicable to the other implementations.

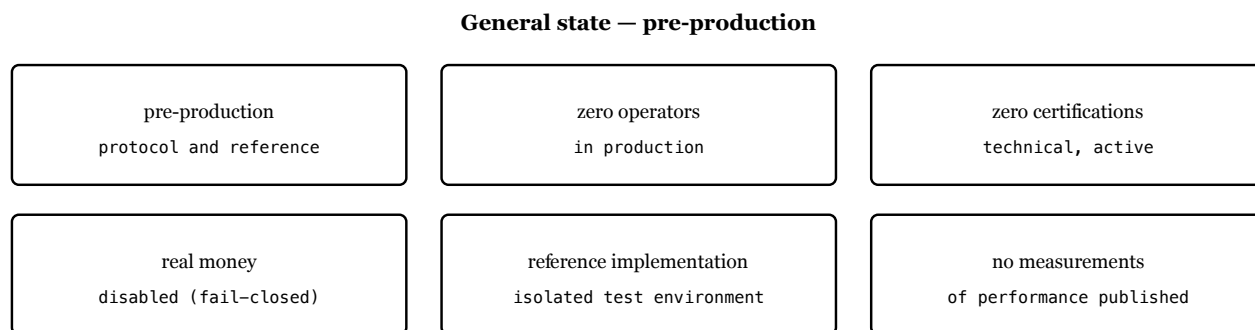


Figure 12. BANZA’s pre-production state: zero production operators, zero active technical certifications, real money disabled and a reference implementation in a test environment.

12 Conclusions

This work presented BANZA as an open financial interoperability protocol based on public, versioned rules, deterministic validation and verifiable evidence. The protocol complements existing interoperability with a common technical base that can be inspected, reproduced and verified by third parties.

The main contributions are:

- evaluation of implementations against public versions and profiles, rather than entities in the abstract;
- separation between the protocol, certification and operational schemes, keeping technical conformance, admission and regulatory authorisation distinct;
- deterministic validation bound to the input artifacts through cryptographic digests, reason codes, receipts and verifiable evidence;
- cumulative conformance profiles and a validation journey with explicit statuses and fail-closed behaviour;
- separation between deterministic decision and BanzAI-assisted explanation.

The guarantees remain limited to the observed technical scope. An implementation's conformance does not constitute authorisation to operate, and the current pre-production state does not yet support conclusions about performance, scalability or adoption in real operational environments.

Future work should focus on measuring reproducibility across independent evaluators and on evaluating the protocol in multi-operator scenarios. It should also analyse performance, scalability, integration costs and the evolution of the conformance profiles.

In summary, BANZA establishes an open, reproducible foundation for financial interoperability, making verifiable the rules, the conformance and the evidence that support each implementation. Its central contribution is to allow technical interoperability to be demonstrated and reproduced by third parties.

References

- International Organization for Standardization. ISO 20022 — Financial services — Universal financial industry message scheme. ISO. <https://www.iso20022.org/>
- International Organization for Standardization. ISO 8583:2023 — Financial-transaction-card-originated messages — Interchange message specifications. Edition 3. ISO, 2023. <https://www.iso.org/standard/79451.html>
- EMVCo, LLC. EMV QR Code Specification for Payment Systems (EMV QRCPs): Merchant-Presented Mode, Version 1.1. EMVCo, November 2020. <https://www.emvco.com/>
- International Organization for Standardization / International Electrotechnical Commission. ISO/IEC 9646-1:1994 — Information technology — Open Systems Interconnection — Conformance testing methodology and framework — Part 1: General concepts. ISO/IEC, 1994.
- National Institute of Standards and Technology (NIST). Secure Hash Standard (SHS). FIPS PUB 180-4. NIST, August 2015. DOI 10.6028/NIST.FIPS.180-4. <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf>
- S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, IRTF, January 2017. DOI 10.17487/RFC8032. <https://www.rfc-editor.org/rfc/rfc8032>
- M. Nottingham. Well-Known Uniform Resource Identifiers (URIs). RFC 8615, IETF, May 2019. DOI 10.17487/RFC8615. <https://www.rfc-editor.org/rfc/rfc8615>
- R. Peng. Reproducible Research in Computational Science. Science, 334(6060):1226–1227, 2011. DOI 10.1126/science.1213847. <https://doi.org/10.1126/science.1213847>